

第一章 计算机系统概论

一、计算机系统简介

1、计算机的软硬件概念

计算机系统：

(1) 硬件：计算机的实体，如主机、外设等。

(2) 软件：由具有各类特殊功能的信息（程序）组成

①系统软件：用来管理整个计算机系统

语言处理程序、操作系统、服务性程序、数据库管理系统、网络软件

②应用软件：按任务需要编制成的各种程序

2、计算机系统的层次结构



汇编程序 (Assembler)：将汇编语言程序翻译成对应的目标程序。

编译程序 (Compiler)：将高级语言程序翻译成对应的目标程序。

链接程序 (Linker)：将一个或多个目标程序与一个或多个相关的程序库 (Library) 组织在一起，产生可执行代码，并存入外存，在需要运行时再由操作系

统加载后执行。

解释程序 (Interpreter)：对高级语言程序逐句地进行翻译，产生对应的机器语言指令序列并执行之。

计算机唯一能直接执行的语言是机器语言。

二、计算机的基本组成

1、冯·诺依曼计算机的特点

冯·诺依曼计算机以“存储程序”为基础，特点为：

(1) 五大组成部件：**运算器、存储器、控制器、输入设备、输出设备**；

(2) 指令和数据以同等地位存于存储器，可按地址寻访；

(3) 指令和数据用二进制表示；

(4) 指令由操作码和地址码组成；

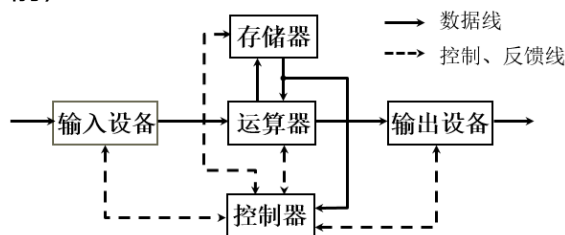
(5) 指令在存储器内按顺序存放（存储程序）；

(6) 以运算器为中心。

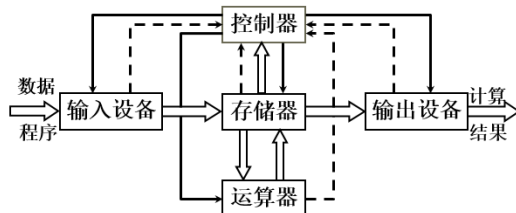
世界上第一台冯·诺伊曼结构的计算机EDVAC是1949年诞生的。

2、计算机硬件框图

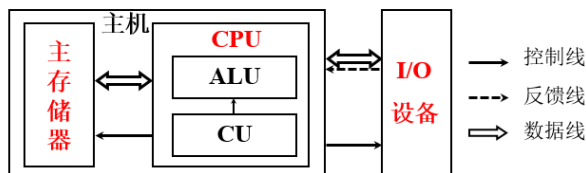
(1) 典型冯·诺依曼计算机以**运算器**为中心（改进前）：



改进后：以**存储器**为中心



(2) 现代计算机结构框图：现代计算机以存储器为中心

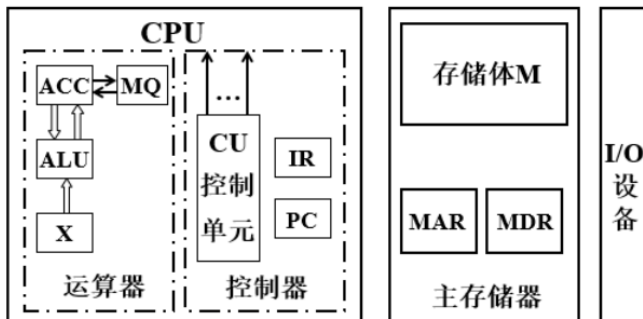


如何区分从存储器中取出的是指令还是数据？

计算机硬件主要通过不同的时间段来区分指令和数据，即：取指周期（或取指微程序）取出的既为指令，执行周期（或相应微程序）取出的既为数据。

另外也可通过地址来源区分，从PC指出的存储单元取出的是指令，由指令地址码部分提供操作数地址。

3、计算机的工作步骤



(1) 存储器的基本组成



MAR：存储器地址寄存器，反映存储单元的个数；

MDR：存储器数据寄存器，反映存储字长。

存储单元：存放一串二进制代码/存放一个存储字的所有存储元集合

存储字：存储单元中二进制代码的组合

存储字长：存储单元中二进制代码的位数，往往是8

字节的整数倍

每个存储单元赋予一个地址号

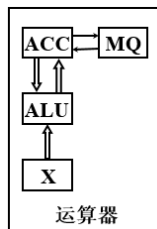
按地址寻访

存储容量：存储器内二进制信息的位数

计算机存储数据的基本单位为字节。

(2) 运算器的基本组成及操作过程

运算器：是计算机的数据处理中心，完成各种算术运算、逻辑运算、移位操作等功能。



ACC：累加器

MQ：乘商寄存器

X：操作数寄存器

	ACC	MQ	X
加法	被加数、和		加数
减法	被减数、差		减数
乘法	乘积高位	乘数、乘积低位	被乘数
除法	被除数、余数	商	除数

ALU：算术逻辑单元

①加法操作过程

初态：ACC 被加数 $[M] \rightarrow X$ $[ACC] + [X] \rightarrow ACC$

②减法操作过程

初态：ACC 被减数 $[M] \rightarrow X$ $[ACC] - [X] \rightarrow ACC$

③乘法操作过程

初态：ACC 被乘数 $[M] \rightarrow MQ$ $[ACC] \rightarrow [X]$ $0 \rightarrow ACC$ $[X] \times [MQ] \rightarrow ACC // MQ$

④除法操作过程

初态：ACC 被除数 $[M] \rightarrow X$ $[ACC] \div [X] \rightarrow MQ$ 余数在ACC中

(3) 控制器的基本组成

功能：控制指令的读出、解释和执行，中断事件的处

理等。

组成：程序计数器PC—存放当前待执行指令所在地址；

指令寄存器IR—寄存当前的指令（现行的指令）；

控制单元CU—解释现行指令。

程序：有序指令的集合，用来解决某一特定问题。

指令：机器完成某种操作的命令，包括操作码和地址码。

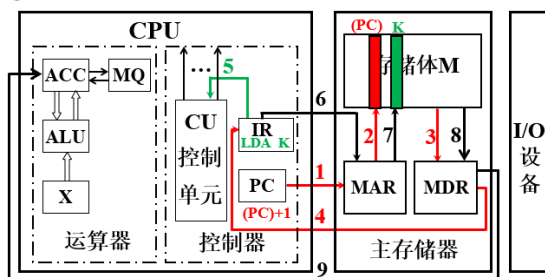
一条指令的执行过程：取指令—分析指令—执行指令。

PC 存放当前欲执行指令的地址，具有计数功能
(PC) + 1 → PC；

IR 存放当前欲执行的指令。

指令的执行过程（信息流向）：取指令→分析指令→执行指令。

①取数指令

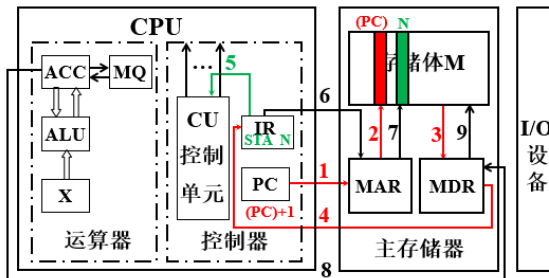


15. 已知某顺序执行程序程序中有一条取数指令，该指令保存在内存23H单元中。以取数指令为例，分析指令执行过程：

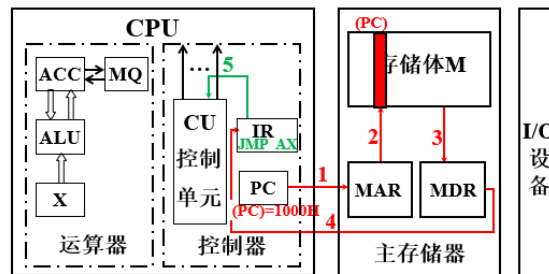
- ① 取指令 阶段：PC或23H → MAR → 存储体 → MDR → IR
- ② 分析指令 阶段：OP(IR) → CU
- ③ 执行指令 阶段：Ad(IR)或N → MAR → 存储体 → MDR → ACC

同时，形成下一条指令的地址，(PC) 或23H + 1 → PC

②存数指令：STA N



③跳转指令：JMP AX，寄存器(AX)=1000H



三、计算机硬件的主要技术指标

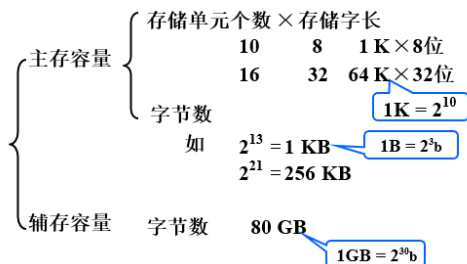
1、机器字长

CPU一次能处理的数据的位数，通常与CPU中寄存器的位数有关。

字长位数越多，精度越高，运算速度越快；通常是8的整倍数，使能存放整数个字符的编码。

机器字长、指令字长和存储字长可以相等也可以不相等。

2、存储容量：存放二进制信息的总位数。



3、运算速度

(1) 平均运算速度 V_m (指令平均执行速度)

定义：计算机每秒钟执行指令的条数；单位：MIPS (百万条指令每秒)；

求法：先求平均运算时间 (平均指令周期) T_m

式中：n：指令的种类；：第i种指令出现的频度(%)
：第i种指令的指令周期(秒)

则平均运算速度： $V_m = 1 / T_m$

(2) 指令的平均周期数CPI

定义：计算机执行一条指令所需的时钟周期数。

求法：

相关参数：IPC(CPU 每一时钟周期内所执行的指令条数)

说明：一旦处理器设计完成，IPC值就不会再改变了
平均运算速度 V_m 与平均周期数CPI的关系：

(3) 每秒浮点运算次数FLOPS

定义：每秒所执行的浮点运算次数。

第二章 计算机的发展及应用

1、计算机发展的阶段 (按照采用的电子器件划分)：

第一代：电子管计算机；第二代：晶体管计算机；第三代：中、小规模集成电路计算机；第四代：大规模集成电路计算机；第五代：超大规模集成电路计算机。

2、计算机的应用

CAD：计算机辅助设计；CAM：计算机辅助制

造；CAI：计算机辅助教学；
CIMS：计算机/现代集成制造系统。
办公自动化和管理信息系统、虚拟现实、人工智能、
多媒体技术。

第三章 系统总线

一、总线的基本概念

- 1、总线是连接多个部件的公共信息传输线，是各个部件**分时、共享**的传输介质。
- 2、同一时刻仅允许一个部件向总线发送信息，多个部件可同时接受相同的信息。

二、总线分类

按**数据传送方式**不同,总线可分为串行传输总线和并行传输总线。

系统总线是用来连接CPU、主存和外设部件。按**连接部件**不同,总线通常可分为片内总线、系统总线和控制总线三种。

1、片内总线—芯片内部的总线

CPU内部，寄存器与寄存器之间、寄存器与算术逻辑单元之间都由片内总线连接。

2、系统总线—指计算机各部件之间的信息传输线

按**传输信息**的不同,系统总线通常可分为数据总线、地址总线和控制总线三种。

(1) 数据总线

双向传输、与机器字长、存储字长有关。传输的是指令，数据，中断类型（入口地址）

若数据总线宽度为8位，指令字长为16位，则CPU取指阶段必须两次访问主存。

(2) 地址总线

单向传输、与存储地址、I/O地址有关。传输的是内存地址、I/O设备地址（端口号）

(3) 控制总线

有出（中断请求、总线请求）、有入（存储器读、存储器写、总线允许、中断确认）。

对象不同，方向不同。 并行总线传输速度并不一定比串行总线快

3、通信总线

用于计算机系统之间或计算机系统与其他系统之间的通信。

按传输方式分为串行通信总线和并行通信总线。

(1) 串行通信总线：使用一条传输线，采用脉冲传送；

优点：只需要一条传输线，成本比较低廉。 缺点：速度慢。

(2) 并行传送：每一位数据需一条传输线，一般采用电位传送；

优点：短距离内，传输速度快。缺点：成本高；距离短。

三、总线性能指标

总线特性：机械特性、电气特性、功能特性、时间特性。

1、总线宽度

数据线的根数，用位表示。

2、总线带宽

每秒传输的最大字节数 (MBps)；最大数据传输率 = 总线宽度 × 总线工作频率。

总线工作频率为 33MHz，总线宽度为 32 位，则总线带宽为 $33 \times (32 \div 8) = 132 \text{MBps}$ 。

3、总线复用

一种信号线在不同的时间传输不同的信息，可以减少总线中信号线的数量。

不同信号共用一组信号线,分时传送,这种总线传输方式是总线复用。

四、总线控制

总线控制：统一管理总线上多个部件如何发送信息，如何接收信息，如何防止信息丢失等一系列问题。

1、判优控制—解决总线使用权问题

当多个设备同时请求占用总线时，由总线判优控制器按其优先级别仲裁，决定由哪个主设备占用总线。主设备：对总线有控制权，可向总线发送请求。从设备：对总线没有控制权，只能响应主设备发来的总线命令。

(1) **集中式—将控制功能集中在一处（如在CPU中）** **BS 信号的建立者是** 获得总线使用权的设备

①链式查询（对电路故障最敏感）

优点：需3根控制线，线路少，优先级固定，易扩展；

缺点：优先级低的设备可能难获得总线使用权，对电路故障敏感。

在链式查询方式下，若有N个设备，则只有一条总线请求线。

②计数器定时查询

优点：设备的优先级可改变，系统故障敏感度降低；

缺点：需 $(\log_2 N + 2)$ 条控制线，线路复杂。其中N为设备总数。

在计数器定时查询方式下，若计数从0开始，则设备号小的优先级高。

计数器定时查询，若每次计数从上一次计数的终止点开始，则每个设备使用总线的机会相等。

③独立请求方式（响应时间最快）

优点：设备的优先级可灵活改变，响应速度快；

缺点：控制线数量多 $(2N)$ ，结构复杂。

(2) 分布式—将控制功能分散在各个部件或设备上
2、通信控制—解决获得使用权后，通信双方协调配合问题

目的：主要解决通信双方如何获知传输开始和传输结束、以及通信双方如何协调如何配合。

总线周期：完成一次总线操作的时间。（申请分配阶段、寻址阶段、传数阶段、结束阶段）

(1) 同步通信—由统一时钟控制数据的传送

同步控制方式是由统一的时序信号控制的方式。

在同步通信中,一个总线周期的传输过程是先传送地址,再传输数据。

优点:规定明确、统一,模块间配合简单一致。

缺点:主从模块的时间配合属于“强制性同步”,对于速度不同的部件而言,严重影响总线工作效率,缺乏灵活性。

适用场合:总线长度短(短距离)、各部件存取时间一致(或差异不大)的场合。

(2) 异步通信—没有公共时钟标准,采用应答方式

应答方式:又称握手方式,即主模块发出请求,从模块响应,然后开始通信。

特点:没有同步时钟,允许不同速度的模块和谐工作。但是结构复杂,且对传送的字符有格式要求。

格式:1个起始位(低电平),5~8个数据位,1个奇偶校验位(校验位+信息位),1或1.5或2个终止位(高电平)。起始位后紧跟传送字符的最低位。

①不互锁(速度最快) 波特率为单位时间内传输的二进制数据的位数

主设备请求信号的撤除与是否收到应答无关;从设备应答信号的撤除与主设备是否撤除请求信号无关。

举例:单机系统中,CPU向主存写信息,所给出的地址、数据、写命令以及写入数据。

②半互锁

主设备请求信号的撤除必须在收到应答信号后;从设备应答信号的撤除与主设备是否撤除请求信号无关。

举例:多机系统中,CPU访问共享存储器,发出访存请求后,必须收到未占用的应答信号方可进行访存。

③全互锁

主设备请求信号的撤除必须在收到应答信号后;

从设备应答信号的撤除必须得到主设备撤除请求信号后。

举例：网络通信。

(3) 半同步通信—同步、异步结合（即采用时钟信号，又采用握手信号）

特点：既有公共时钟控制，又允许速度不同的工作模块协调的工作，只需插入等待周期的措施来协调通信双方的配合问题，称作半同步控制。

优点：控制方式比异步通信简单，可靠性高。

缺点：对系统时钟频率不能要求太高，系统工作速度不高。总线传输周期变长。

(4) 分离式通信—充分挖掘系统总线每个瞬间的潜力

特点：充分提高了总线的有效占用。

在分离式通信方式中，总线上所有模块都可以成为主模块。

第四章 存储器

一、存储器分类

计算机的存储系统是指Cache、主存储器和外存储器。

计算机的存储器采用分级是为了解决容量、速度、价格三者之间的矛盾。

1、主存

(1) 随机存储器RAM—在程序的执行过程中可读可写

主要用于：主存，高速缓冲存储器

3. 动态 RAM 和静态 RAM 的比较

	DRAM	SRAM
□ 存储原理	电容	触发器
□ 集成度	高 (4-8倍)	低
□ 芯片引脚	少	多
□ 功耗	小	大
□ 价格	低	高 (8-16倍)
□ 速度	慢	快 (8-16倍)
□ 刷新	有	无

① 静态RAM (SRAM)

存储原理：用触发器工作原理存储信息。

特点：信息读出后，仍保持原状态，不需要再生；断电后，原信息丢失。

存取速度快，但集成度低，功耗大，一般用于组成高速缓冲存储器。

某一RAM芯片，容量为1K×4位，其数据线为4条，地址线为10条。若重合法驱动方式，其字线条数为80条。

② 动态RAM (DRAM)

存储原理：电容存储电荷的原理存储信息。

特点：所存储信息需要刷新；集成度高，功耗更低。

DRAM的刷新是以行为单位的。

常见的动态RAM基本单元电路有三管式和单管式两种，它们共同的特点都是靠电容存储电荷的原理来寄存信息。

动态半导体存储器需要刷新是因为存储电荷的电容放电。

刷新（间隔、死区）

实质：将原存信息读出，再经由刷新放大器形成原信息并重新写入的过程。

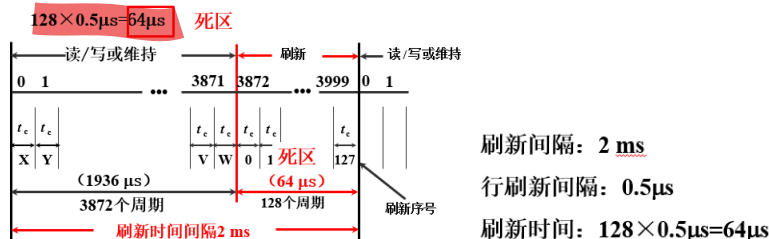
刷新原因：电容上的电荷一般只能维持1~2ms，即使不断电，信息也会自动消失。如果存储单元长期得

不到访问，则其原信息将会慢慢消失。

动态 RAM 刷新方式：

a. **集中刷新**：在规定的的一个刷新周期（2ms）内，对全部存储单元集中一段时间逐行进行刷新，刷新时必须停止读/写操作。

□ 举例：128×128矩阵，存取周期为0.5 μs，刷新时间为



优点：读写操作时不受刷新工作的影响，因此系统的存取速度比较高。

缺点：在集中刷新期间（死区）不能进行读写，而且存储容量越大，死区就越长。这增加了存储管理的困难，显然对于高速高效的计算机系统工作是不利的。

b. **分散刷新（不存在死时间）**

把刷新操作分散到每个存取周期内进行。

优点：**没有死区**。

缺点：加长了系统的存取周期，降低了整机速度；刷新过于频繁，从而使刷新时间增加。没有充分利用所允许的最大时间间隔（2ms）。

c. **异步刷新**

前两种方式的结合，充分利用了最大刷新间隔时间，把刷新操作平均分配到整个最大刷新间隔时间内进行。故：**相邻两行的刷新间隔 = 最大刷新间隔时间 ÷ 行数**

优点：**死区小，刷新占用时间少**。

缺点：**系统结构变复杂**。

(2) 只读存储器ROM—在程序的执行过程中只读

主要用于：BIOS芯片，U盘

- a. 掩模 ROM (MROM, 不可改写)
- b. PROM (可编程只读存储器，一次性编程)
- c. EPROM (可擦除可编程只读存储器，多次性编程)

EPROM是可改写的,但不能作为随机存储器

- d. EEPROM (电擦除只读存储器，多次性编程)
- e. Flash Memory (闪速型存储器) U盘

(3) 技术指标

存储容量=存储单元个数 × 存储字长

存取周期=存取时间+复原时间

存储器的存取周期是存储器进行连续读或写操作所允许的最短时间间隔。

存取时间：启动一次存储器操作到完成该操作所需的全部时间；

读出时间：存储器收到有效地址开始，到产生有效输出；

写入时间：存储器收到有效地址开始，到数据写入被选中单元。

存储器的带宽是单位时间内存储器存取的信息量；单位：位/秒或字节/秒。

提高存储器带宽的方法：缩短存取周期、增大存储字长、增加存储体。

2、辅存

磁盘是直接存取，磁带是顺序存取。

3、缓存

将主存块调入缓存的任务全部由硬件完成。

4、闪存

闪存是EEPROM的进一步发展,可读可写，用MOS管的浮栅上有无电荷来存储信息。闪存是ROM的一种，写入时必须先擦除原有数据,所以写速度比读速度慢很多。闪存是一种非易失存储器，采用随机访问方式。现在常见的SSD固态硬盘就由Flash芯片构

成。

存储元由MOS管组成是一种半导体存储器；掉电后信息不易失，是一种非易失性存储器；采用随机访问方式，可替代计算机外部存储器。

二、存储器的层次结构

1、主存-辅存层次

主存：速度快、容量小、价格高；辅存：速度慢、容量大、价格低。

2、主存-缓存层次

主存和缓存按块存储，块的大小相同，块内地址相同。

三、主存中地址单元的分配：地址码能够指定的最小存储单位

1、按字编制—访问的单位为字长

计算机可寻址的最小信息单位是一个存储字，一个存储字所包含的二进制位数称为存储字长。

某机字长32位，存储容量为4MB，若按字编址，它的寻址范围是1M。

2、按字节编制—访问的单位为一个字节

计算机可寻址的最小信息单位是一个字节。

某机字长16位，存储容量为64KB，若按字节编址，它的寻址范围是0-(64K-1)。

四、半导体存储芯片的译码驱动方式

1、线选法

特点：一根字选择线可直接选中一个存储单元的各位。结构简单，但只适用于容量不大的存储芯片。

某一RAM芯片，容量为1K×1位，其数据线为1条，地址线为10条。若采用线选法驱动方式，其位线条数为1条，其字线条数为1K或1024条。

2、重合法

被选单元由X和Y两个方向的地址决定，故称为重合法。

某一RAM芯片，容量为1K×1位，其数据线为1条，

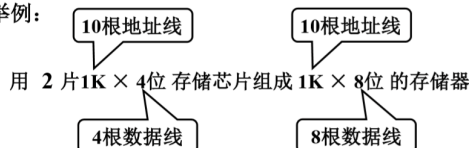
地址线为10条。若重合法驱动方式,其位线条数为1条,其字线条数为64条。

五、存储器的CPU的连接

1、位扩展（增加存储字长）

位扩展：只在**位数**（与数据线有关）方向扩展，而芯片的字数（与地址线有关）和存储器的字数一致。

举例：

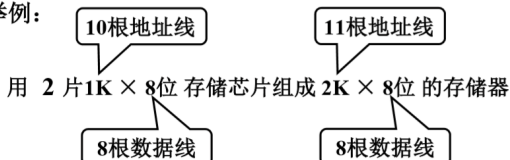


$$\text{总片数} = \frac{\text{总容量}}{\text{容量/片}} = \frac{1\text{K} \times 8}{1\text{K} \times 4} = 2$$

2、字扩展—增加存储字（存储单元）的数量

字扩展：只在**字数**（与地址线有关）方向扩展，而芯片的位数（与数据线有关）和存储器的位数一致。

举例：

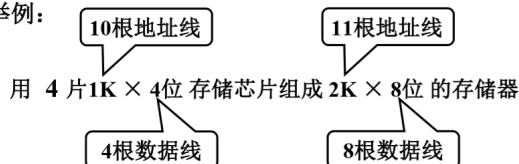


$$\text{总片数} = \frac{\text{总容量}}{\text{容量/片}} = \frac{2\text{K} \times 8}{1\text{K} \times 8} = 2$$

3、位、字扩展

位和字扩展：在**字数**和**位数**方向同时扩展（既增加存储器的字的数量又增加字长）。

举例：

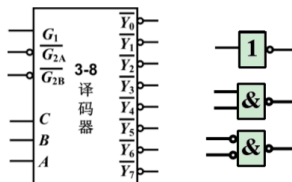


$$\text{总片数} = \frac{\text{总容量}}{\text{容量/片}} = \frac{2\text{K} \times 8}{1\text{K} \times 4} = 4$$

例4.1 设CPU有16根地址线，8根数据线，用MREQ作访存控制。现有下列芯片：1K×4位RAM；4K×8位RAM；8K×8位RAM；2K×8位ROM；4K×8位ROM；8K×8位ROM及74LS138等电路

要求：构成地址为**6000~67FFH**的系统程序区；

地址为**6800~6BFFH**的用户程序区，选择芯片并画出逻辑连接图。



例4.1 解： **地址为6000~67FFH的系统程序区**
 地址为6800~6BFFH的用户程序区

(1) 写出地址范围对应的二进制地址码，并确定总容量

$A_{15}A_{14}A_{13}A_{12}$	$A_{11}A_{10}A_9A_8$	$A_7A_6A_5A_4$	$A_3A_2A_1A_0$	
0 1 1 0	0 0 0 0	0 0 0 0	0 0 0 0	1片2K×8位 ROM 2K×8位
⋮				
0 1 1 0	0 1 1 1	1 1 1 1	1 1 1 1	
0 1 1 0	1 0 0 0	0 0 0 0	0 0 0 0	1K×8位 RAM 2片1K×4位
⋮				
0 1 1 0	1 0 1 1	1 1 1 1	1 1 1 1	

• **系统程序区：**需要的容量为2K×8位，根据其特点，
选择ROM。

所以选择1片2K×8位的ROM，不需扩展。

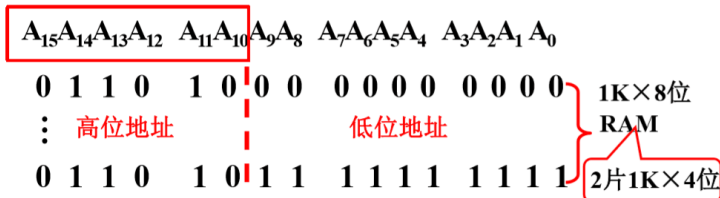
• **用户程序区：**需要的容量为1K×8位，根据其特点，
选择RAM。

所以选择2片1K×4位的RAM，需位扩展。

(3) 分配CPU的地址线

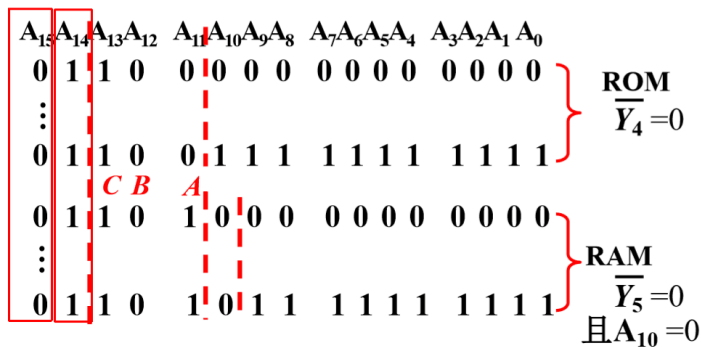


∴ CPU的 $A_0 \sim A_{10}$ 分别连接ROM的 $A_0 \sim A_{10}$, CPU的 $A_{11} \sim A_{15}$ 做其片选信号。



∴ CPU的 $A_0 \sim A_9$ 分别连接RAM的 $A_0 \sim A_9$, CPU的 $A_{10} \sim A_{15}$ 做片选信号

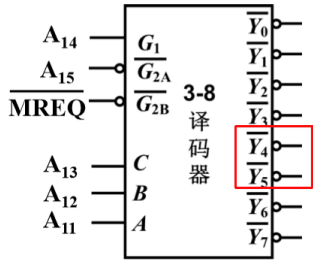
(4) 确定片选信号



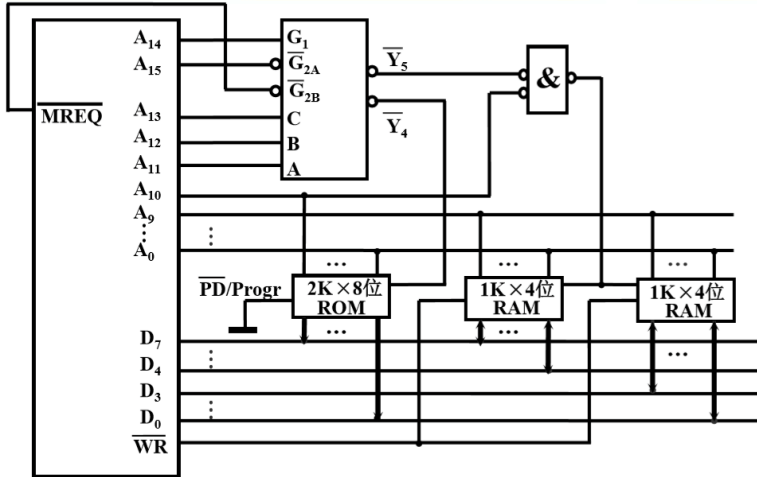
A_{15} 可以连 $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$

A_{14} 可以连 G_1

根据74138的工作方式：



- G_1 接高电平，与 A_{14} 相连；
- $\overline{G}_{2A}$ 和 $\overline{G}_{2B}$ 接低电平，可分别与 A_{15} 和 $\overline{MREQ}$ 相连；
- $A_{13} A_{12} A_{11}$ 分别与CBA相连；
- $\overline{Y}_4$ 作为ROM的片选信号；
- $\overline{Y}_5$ 和 A_{10} 同时为0时，选中RAM



六、汉明码

1、具有一位纠错能力

任意两组合法代码之间 二进制位数的 **最少** 差异

编码的纠错、检错能力与编码的最小距离有关

$$L - 1 = D + C \quad (D \geq C)$$

L —— 编码的最小距离

$$L = 3$$

D —— 检测错误的位数

具有一位 纠错能力

C —— 纠正错误的位数

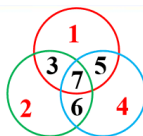
2、汉明码构成

2. 汉明码的组成

- 汉明码的分组是一种交叉式非划分方式。

假设采用奇校验的汉明码：

1	2	3	4	5	6	7
---	---	---	---	---	---	---



思路：分成3组，每组有1位校验位，共3位奇校验位。4位信息位。

若接收到的汉明码为：

0	1	1	0	0	1	1
---	---	---	---	---	---	---

思考：如何判断是否出错？

红圈内成员1,3,5,7的值为：**0101** 有错

绿圈内成员2,3,6,7的值为：**1111** 有错 则3组共有的7出错。

蓝圈内成员4,5,6,7的值为：**0011** 有错

计算机用异或运算进行奇偶检验码的检错。

例4.4 求 0101 按“偶校验”配置的汉明码

解：∵ $n = 4$ 根据 $2^k \geq n + k + 1$ 得 $k = 3$

汉明码排序如下：

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	1	0	1
	0	1		0			

$$C_1 = 3_{\text{位}} \oplus 5_{\text{位}} \oplus 7_{\text{位}} = 0 \oplus 1 \oplus 1 = 0$$

$$C_2 = 3_{\text{位}} \oplus 6_{\text{位}} \oplus 7_{\text{位}} = 0 \oplus 0 \oplus 1 = 1$$

$$C_4 = 5_{\text{位}} \oplus 6_{\text{位}} \oplus 7_{\text{位}} = 1 \oplus 0 \oplus 1 = 0$$

∴ 0101 的汉明码为 **0100101**

练习：按配奇原则配置 0011 的汉明码。

解：∵ $n = 4$ 根据 $2^k \geq n + k + 1$ 得 $k = 3$

汉明码排序如下：

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	0	1	1
	0	1		1			

$$C_1 = 3_{\text{位}} \oplus 5_{\text{位}} \oplus 7_{\text{位}} = 0 \oplus 0 \oplus 1 = 0$$

$$C_2 = 3_{\text{位}} \oplus 6_{\text{位}} \oplus 7_{\text{位}} = 0 \oplus 1 \oplus 1 = 1$$

$$C_4 = 5_{\text{位}} \oplus 6_{\text{位}} \oplus 7_{\text{位}} = 0 \oplus 1 \oplus 1 = 1$$

∴ 0101 的汉明码为 **0101011**

例4.5 已知接收到的汉明码为 **0100111**，按配偶原则配置。

试问要求传送的信息是什么？

解：纠错过程如下

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7 = 0 \oplus 0 \oplus 1 \oplus 1 = 0 \quad \text{无错}$$

$$P_2 = 2 \oplus 3 \oplus \boxed{6} \oplus 7 = 1 \oplus 0 \oplus 1 \oplus 1 = 1 \quad \text{有错}$$

$$P_4 = 4 \oplus 5 \oplus \boxed{6} \oplus 7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1 \quad \text{有错}$$

$$\therefore P_4 P_2 P_1 = 110$$

第6位出错，可纠正为 **0100101**，故要求传送的信息为 **0101**。

练习：写出按偶校验配置的汉明码 **1100101** 的纠错过程。

解：纠错过程如下

$$P_1 = \boxed{1} \oplus 3 \oplus 5 \oplus 7 = 1 \oplus 0 \oplus 1 \oplus 1 = 1 \quad \text{有错}$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7 = 1 \oplus 0 \oplus 0 \oplus 1 = 0 \quad \text{无错}$$

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0 \quad \text{无错}$$

$$\therefore P_4 P_2 P_1 = 001$$

第1位出错，可不纠正。故要求传送的信息为 **0101**。

1. 欲使用汉明码传送**4位**信息**0101**，则需添加 **3** 位校验位。若汉明码从左到右分别为第1位、第2位.....则检测位的分别为第 **1** 位、第 **2** 位、第 **4** 位。

2. 欲使用汉明码传送4位信息0101。若按偶校验进行配置，则汉明码是 **0100101** 若按奇校验进行配置，则汉明码是 **1001101**

3. 若接收到(7,4)位汉明码，需形成新的检测位 P_i 。汉明码从左到右的位置分别为1、2、.....、7。则

$$P_1 = 1 \oplus \boxed{3} \oplus \boxed{5} \oplus \boxed{7};$$

$$P_2 = 2 \oplus \boxed{3} \oplus \boxed{6} \oplus \boxed{7};$$

$$P_4 = 4 \oplus \boxed{5} \oplus \boxed{6} \oplus \boxed{7}。$$

若汉明码采用偶检验，则 $P_4 P_2 P_1 =$ **000** 时，无错。

若 $P_4 P_2 P_1 = 110$ 时，则第 **6** 位出错。



4. 若接收到(7,4)位汉明码, 需形成新的检测位 P_i 。汉明码从左到右的位置分别为1、2、.....、7。则

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7;$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7;$$

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7。$$

若汉明码采用奇检验, 则 $P_4 P_2 P_1 = 111$ 时, 无错。

若 $P_4 P_2 P_1 = 110$ 时, 则第 1 位出错。

七、高位交叉和低位交叉

(1) 高位交叉: 高位体号, 低位体内地址, 适合存储器容量的扩展。

高位交叉编址的特点:

- 通过多体并行提高访存速度, 有利于存储器容量的扩展。
- 若采用高位交叉编址, 则连续读取 n 个字所需时间为 nT (T 为存取周期)。

5. 一个四体存储器, 每个模块容量是 $64K \times 32$ 位, 存取周期为200ns。若存储器采用高位交叉编址, 在一个存取周期中, 存储器能向CPU提供 32 位二进制信息。

(填写阿拉伯数字)

(2) 低位交叉: 结合流水线技术, 适合提高存储器带宽和访问速度。低位是体号, 高位是体内地址

3. 一个四体并行低位交叉存储器, 每个模块的容量是 $64K \times 32$ 位, 存取周期为200ns, 在下述说法中()是正确的。 B

- A 在200ns内, 存储器能向CPU提供256位二进制信息
- B 在200ns内, 存储器能向CPU提供128位二进制信息

5. 一个四体存储器, 每个模块容量是 $64K \times 32$ 位, 存取周期为200ns。若存储器采用低位交叉编址, 在一个存取周期中, 存储器能向CPU提供 128 位二进制信息。

6. 一个**四体**低位交叉存储器，存取周期为 T ，总线传输周期为 τ ，为实现流水线方式存取，应满足 $T =$ **4** τ 。

为保证第二次启动某个存储模块，则该模块的上一次存取操作必须已经完成，要求存储器模块的数量应该**大于等于 4**]。(填写阿拉伯数字)

- 高位交叉存储器有利于存储器容量的扩展，提高访存速度的原因是多体并行

设 **n 体高位**交叉存储器，存取周期为 **T** ，总线传输周期为 **τ** ，则连续读取 **n** 个字所需的时间是： **$n T$**

- 低位交叉存储器有利于增加存储器带宽，提高访存速度的原因是结合流水线技术，提高带宽。

设 **n 体低位**交叉存储器，存取周期为 **T** ，总线传输周期为 **τ** ，则连续读取 **n** 个字所需的时间是： **$T + (n-1) \tau$**

八、高速缓存

目的：解决CPU和主存之间的速度匹配问题。

1、基本原理：程序访问局部性原理。

局部性原理：指令和数据在主存内都是连续存放的，且有些指令和数据往往会被多次调用（如子程序、循环程序和一些常数），即指令和数据在主存的地址分布不是随机的，而是相对的簇聚，使得CPU在执行程序时，访存具有相对的局部性。

根据程序访问的局部性原理，只要将**CPU近期要用到的程序和数据**提前从主存送到Cache中，就可以做到CPU在一定时间内只访问Cache。

2、命中率

缓存容量越大，命中率越高。命中率=总命中次数/（总命中次数+访问总次数）

随1. Cache用 [**标记**] 记录与某缓存块建立了对应关系的主存块号；Cache的命中率与Cache的**容量** [**2**] 与 [**块长**] 有关。

3、地址映射

6. 将主存地址映射到缓存中定位称为**地址映射**, 将主存地址变换成缓存地址称为**地址变换**, 当新的主存块需要调入缓存中, 而它的可用位置又被占用时, 需根据**替换算法**解决调入问题。 **A**

随2. Cache的写操作比较复杂, 需要保持Cache与主存内容一致。目前主要采用的方法有写直达法和写回法。速度较快的是 **写回法**, 因为其写操作时间就是访问Cache的时间, 但是在读Cache失效且需要发生数据块替换时, 需要将替换的块一次写回主存, 增加了Cache的复杂性。比较容易实现的是 **写直达法**, 其速度较慢, 因为其写操作时间就是访问主存的时间。

(1) 直接映射

每个主存块只能固定的映射到某个缓存块。最不灵活。

【例4.8】假设主存容量为512KB, Cache容量为4KB, 每个字块为16个字, 每个字32位。

(1) Cache地址有多少位? 可容纳多少块?

解: (1) Cache的容量为4KB= 2^{12} B, 则Cache的地址有12位。

又知每个字是32位=4B, 每个字块16个字,

则每个字块的容量是 $16 \times 4B = 64B$

则Cache中共有 $4KB / 64B = 64$ (块)

(2) 主存地址有多少位? 可容纳多少块?

解: (2) 主存的容量为512KB= 2^{19} B, 则主存的地址有19位。

又知每个字是32位=4B, 每个字块16个字,

则每个字块的容量是 $16 \times 4B = 64B$

则主存中共有 $512KB / 64B = 8192$ (块)

(3) 在直接映射方式下, 主存的第几块映射到Cache中的第5块 (设起始字块为第1块) ?

解: (3) Cache共有64块, 主存共有8192块, 直接映射方式下, 映射关系为: $i=j \bmod C$, 其中*i*为缓存块号5, *j*为主存块号, C为缓存块数64。且起始字块为第1块。

$\therefore j$ 为5, $64+5$, $2 \times 64+5$,,

$$(8192/64-1) \times 64+5=2^{13}-64+5$$

(4) 画出直接映射方式下主存地址字段中各段的位数

解: (4) 各段分别如图所示:

每个字块为 $16 \times 32=64\text{B}$, 则字块内地址为*b*=6位($2^6=64$)

缓存共有64块, 则缓存字块地址同样为*c*=6位。

主存地址共有19位, 则主存字块标记为*t*= $19-6-6=7$ 位

主存字块标记	Cache字块地址	字块内地址
--------	-----------	-------

t=7位

c=6位

b=6位

(2) 全相联

每个主存块可以映射到任何一个缓存块。灵活性最高, 成本最高。

【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(1) 在直接映射方式下，设计主存地址格式。

解：

(1) 主存存储单元个数为 $512\text{K} = 2^{19}$ ，则主存地址共19位

块长 $B = 4 = 2^b$ ，则 $b = 2$ 位， $m = 19 - b = 19 - 2 = 17$ 位

Cache中的块数为 $4096/4 = 1024 = 2^c$ ，则 $c = 10$ 位；

主存字块标记 $t = m - c = 17 - 10 = 7$ 位

直接映射方式下，主存地址格式为：

主存字块标记	Cache字块地址	字块内地址
--------	-----------	-------

$t = 7$ 位

$c = 10$ 位

$b = 2$ 位

(2) 在全相联映射方式下，设计主存地址格式。

解：(2) 块长 $B = 4 = 2^b$ ，则 $b = 2$ 位

全相联映射方式下，主存字块标记 $m = 19 - b = 19 - 2 = 17$ 位

全相联映射方式下，主存地址格式为：

主存字块标记	字块内地址
--------	-------

$m = 17$ 位

$b = 2$ 位

(3) 组相联

某一主存块只能映射到某一缓存组中的任意一块。

n 路组相联映射是指每个缓存组中有 n 个缓存块。

(3) 在二路组相联映射方式下，设计主存地址格式。

解：(3) 主存存储单元个数为 $512K=2^{19}$ ，则主存地址共19位

块长 $B=4=2^b$ ，则 $b=2$ 位， $m=19-b=19-2=17$ 位

在二路组相联映射方式下，一组内有2块，则Cache的组数

为 $4096 \div 4 \div 2 = 512 = 2^9$ ，则 $q=9$ 位

主存字块标记 $s=m-q=17-9=8$ 位

二路组相联映射方式下，主存地址格式为：

主存字块标记	组地址	字块内地址
$s=8$ 位	$c=9$ 位	$b=2$ 位

(4) 若主存容量为 $512K \times 32$ 位，块长不变，在四路组相联映射方式下，设计主存地址格式。

解：

(4) 主存容量为 $512K \times 32$ 位，为双字长存储器。在块长不变，字长不变的情况下。主存块数是原来的2倍。即主存块数为

$512K \times 2 \div 4 = 2^{18}$ ，则 $m=18$ 位

(4) 在四路组相联映射方式下，Cache中的组数为：4096/

$(4 \times 4) = 2^8$ ，则 $q=8$ 位。

主存字块标记 $s=m-q=18-8=10$ 位

主存地址格式为：

主存字块标记	组地址	字块内地址
$s=10$ 位	$q=8$ 位	$b=2$ 位

4、替换策略

(1) 先进先出FIFO

(2) 近期最少使用LRU

(3) 随机法

第六章 计算机的运算方法

一、无符号数和有符号数

1、无符号数

M位寄存器存储无符号数的范围是 $[0, 2^M-1]$ 。

2、有符号数

(1) 原码

只变符号位。看小数点还是逗号。

(2) 补码

-8的补码是1000

当真值为正时，补码等于其原码

若真值为负设 $x = -1010$ 时

$$\begin{aligned} \text{则 } [x]_{\text{补}} &= 2^{4+1} - 1010 = 11111 + 1 - 1010 \\ &= 100000 = 11111 + 1 \\ &\quad - 1010 \quad \quad - 1010 \\ \hline &= 1,0110 \quad \quad \boxed{10101} + 1 \\ &= 1,0110 \end{aligned}$$

$$\text{又 } [x]_{\text{原}} = \boxed{1,1010}$$

当真值为负时，补码可用原码除符号位外每位取反，末位加1求得

(3) 反码

最高位为符号位，书写上用“,”（整数）或“.”

（小数）将数值部分和符号位隔开

对于正数，原码 = 补码 = 反码

对于负数，符号位为1，其数值部分：

原码除符号位外每位取反末位加1 → 补码

原码除符号位外每位取反 → 反码

(4) 移码

$[x]_{\text{移}} = 2^n + x$ 移码和补码只差一个符号位。

二、数的定点表示和浮点表示

1、定点表示

定点机

小数定点机

整数定点机

原码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

补码

$$-1 \sim +(1 - 2^{-n})$$

$$-2^n \sim +(2^n - 1)$$

反码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

移码

$$\underline{\hspace{2cm}}$$

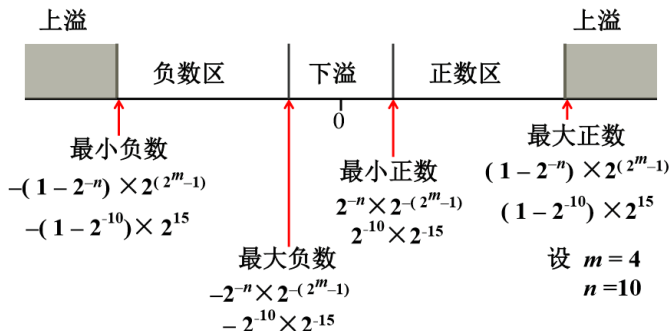
$$-2^n \sim +(2^n - 1)$$

2、浮点表示

2. 浮点数的表示范围

上溢 阶码 > 最大阶码

下溢 阶码 < 最小阶码 按机器零处理



(1) 尾数

真值规格化标准

思考：尾数规格化后浮点数真值的范围

假设尾数数值位 $n=10$ ，阶码数值位 $m=4$ ，阶符、数符各取 1 位。

解：	尾数规格化的真值	尾数未规格化的真值
最大正数	$0.11\ 1111\ 1111 \times 2^{+1111}$ $(1-2^{-10}) \times 2^{15}$	$0.11\ 1111\ 1111 \times 2^{+1111}$
最小正数	$0.10\ 0000\ 0000 \times 2^{-1111}$ $2^{-1} \times 2^{-15}$	$0.00\ 0000\ 0001 \times 2^{-1111}$
最大负数	$-0.10\ 0000\ 0000 \times 2^{-1111}$ $-2^{-1} \times 2^{-15}$	$-0.00\ 0000\ 0001 \times 2^{-1111}$
最小负数	$-0.11\ 1111\ 1111 \times 2^{+1111}$ $-(1-2^{-10}) \times 2^{15}$	$-0.11\ 1111\ 1111 \times 2^{+1111}$

(2) 阶码

$r = 2$ 左规 尾数左移 1 位, 阶码减 1
 右规 尾数右移 1 位, 阶码加 1

$r = 4$ 左规 尾数左移 2 位, 阶码减 1
 右规 尾数右移 2 位, 阶码加 1

$r = 8$ 左规 尾数左移 3 位, 阶码减 1
 右规 尾数右移 3 位, 阶码加 1

基数 r 越大, 可表示的浮点数的范围越大

基数 r 越大, 浮点数的精度降低

当浮点数 尾数为 0 时, 不论其阶码为何值, 按机器零处理

当浮点数 阶码等于或小于它所表示的最小 数 时, 不论尾数为何值, 按机器零处理

(3) 溢出

①上溢: 中断

②下溢: 机器零

(4) IEEE 754标准

【例】分别求二进制数10110010.001的短实数形式的IEEE754标准形式

解: 二进制数: 10110010.001

则 二进制浮点表示为: $1.0110010001 \times 2^{111}$

短实数表示: 总位数为32位, 包含

1位符号位, 8位阶码(含1位阶符), 23位尾数

则: 符号位为 0

阶码: 偏移量为7FH (127), 即1111111, 则偏移后的阶码为:

00000111+01111111=10000110

尾数: 0110010001000000000000

二进制数**1000 1111 1110 1111 1100 0000 0000 0000**

① 表示一个补码整数，其十进制值是多少？

解：作为补码整数，其对应的原码是：

1111 0000 0001 0000 0100 0000 0000 0000

∴ 十进制值是 $-(2^{30} + 2^{29} + 2^{28} + 2^{20} + 2^{14})$

② 表示一个无符号整数，其十进制值是多少？

解：作为无符号整数，其十进制值是：

$2^{31} + 2^{27} + 2^{26} + 2^{25} + 2^{24} + 2^{23} + 2^{22} + 2^{21} + 2^{19} + 2^{18} + 2^{17}$
 $+ 2^{16} + 2^{15} + 2^{14}$

③ 作为IEEE754标准的单精度浮点数

解：阶码是**0001 1111**

指数 $e = \text{阶码} - 127 = 0001\ 1111 - 0111\ 1111$

$= -1100000B = -96D$

尾数=**110 1111 1100 0000 0000 0000**

则**1.M** = **1. 110 1111 1100 0000 0000 0000**

= 1.110 1111 11

∴ 单精度浮点数值为： $X = -(1.110\ 1111\ 11) \times 2^{-96}$

三、定点运算

1、移位运算

(1) 逻辑移位

无符号数的移位运算称为逻辑移位。

逻辑移位产生的空位均补“0”。

逻辑移位左移时 { 最高位丢0，无错误
最高位丢1，结果出错

逻辑移位右移时 { 最低位丢0，无错误
最低位丢1，影响精度

(2) 算术移位

算术移位规则 有符号数的移位运算

符号位不变

真值	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移添 0
	反 码	右移添 1
		1

2、加减法运算

(1) 单符号位补码溢出判断

方法：参加操作的 **两个数**（减法时即为被减数和“求补”以后的减数）**符号相同**，其**结果的符号与原操作数的符号不同**，即为溢出。

硬件实现：

最高有效位的进位 $\oplus$ 符号位的进位 = 1 溢出

如
$$\left. \begin{array}{l} 1 \oplus 0 = 1 \\ 0 \oplus 1 = 1 \end{array} \right\} \text{有 溢出} \quad \left. \begin{array}{l} 0 \oplus 0 = 0 \\ 1 \oplus 1 = 0 \end{array} \right\} \text{无 溢出}$$

(2) 双符号位补码溢出判断

结果的双符号位 相同 未溢出

00. × × × × ×

11. × × × × ×

结果的双符号位 不同 溢出

10. × × × × ×

01. × × × × ×

最高符号位 代表其 真正的符号，其余为数值部分。

3、乘法运算

小数原码乘法运算

(1) 原码一位乘运算规则

以小数为例 设 $[x]_{\text{原}} = x_0 \cdot x_1 x_2 \dots x_n$

$$[y]_{\text{原}} = y_0 \cdot y_1 y_2 \dots y_n$$

$$[x \cdot y]_{\text{原}} = (x_0 \oplus y_0) \cdot (0 \cdot x_1 x_2 \dots x_n)(0 \cdot y_1 y_2 \dots y_n)$$

$$= (x_0 \oplus y_0) \cdot x^* y^*$$

式中 $x^* = 0 \cdot x_1 x_2 \dots x_n$ 为 x 的绝对值

$y^* = 0 \cdot y_1 y_2 \dots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相乘 $x^* \cdot y^*$

 例6.17 已知 $x = -0.1110$ $y = -0.1101$ 求 $[x \cdot y]_{\text{原}}$

$$\text{解: } [x \cdot y]_{\text{原}} = (x_0 \oplus y_0) \cdot (0 \cdot x_1 x_2 \dots x_n)(0 \cdot y_1 y_2 \dots y_n)$$

$$= (x_0 \oplus y_0) \cdot x^* y^*$$

$$[x]_{\text{原}} = 1.1110 \quad [y]_{\text{原}} = 1.1101$$

$$\text{① 乘积的符号位 } x_0 \oplus y_0 = 1 \oplus 1 = 0$$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.1110 \cdot 0.1101$$

四、浮点数加减运算

(1) 对阶 对齐小数点，**小阶向大阶看齐**。

(2) 尾数求和 **双符号位的补码形式**进行加减运算。

(3) 规格化 将非规格化的结果值进行**规格化处理**

(4) 结果舍入 尽量保持计算后数据的精确度

(5) 溢出判断 以阶码的溢出为标准

1、对阶

小阶向大阶看齐

2、尾数求和

对阶后的尾数按定点数加减运算规则求和（差）

以**双符号位的补码形式**进行加减法操作

3、规格化

(2) 规格化数的判断

$S > 0$	规格化形式	$S < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	$0.1 \times \times \dots \times$	原码	$1.1 \times \times \dots \times$
补码	$0.1 \times \times \dots \times$	补码	$1.0 \times \times \dots \times$
	$00.1 \times \times \dots \times$		$11.0 \times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同

(4) 左规

尾数出现 $00.0 \times \times \dots \times$ 或 $11.1 \times \times \dots \times$ 时

需左规

尾数左移一位，阶码减 1，直到数符和第一数位不同为止

(5) 右规

当尾数溢出 (>1) 时，需右规

即尾数出现 $01. \times \times \dots \times$ 或 $10. \times \times \dots \times$ 时

尾数右移一位，阶码加 1

4、舍入

(1) 0舍1入

6. 如果采用0舍1入法进行舍入处理,则
0.01010110011舍去最后一位后,结果为()。

- A 0.0101011001
- B 0.0101011010
- C 0.0101011011
- D 0.0101011100

(2) 恒置1

7. 如果采用末位恒置1法进行舍入处理,则

0.01010110011舍去最后一位后,结果为()。

- A 0.0101011001

5、溢出判断

用阶码判断溢出

- (1) 上溢：中断
- (2) 下溢：机器零

5. 溢出判断 在浮点运算中，根据阶码判断是否溢出

- 上溢 阶码 > 最大阶码
- 下溢 阶码 < 最小阶码 按 **机器零** 处理

设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码的数值部分取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为：

最大阶码：00,111 1111 最大尾数：00.11

最小阶码：11,000 0000 最小尾数：11.000 0000

第七章 指令系统

一、机器指令

分类：微指令、机器指令、宏指令。

指令：用二进制代码表示的结构形式，有操作码和地址码两部分组成。

1、操作码—指明该指令所要完成的操作

操作码的位数反映了机器的操作种类。

操作码字段为 8 位，最多包含指令的数目为 $2^8=256$ 条。

(1) 操作码固定

特征：所有指令长度均相同。

优点：控制简单，速度快，适用于字长较长的计算机。

(2) 扩展操作码

操作码的位数随地址数的减少而增加。目的：保持指令字长度不变而增加指令的数量。

特征：利用操作码扩展技术将操作码扩展到地址码字段，使各类指令的操作码长度不同。

优点：有效的压缩操作码的平均长度，适用于字长较短的计算机。

2、地址码

四地址、三地址、二地址、一地址、零地址。

二、数据在存储器中的存放方式

(1) 大端（优先存高字节）

设存储字长32位，存储器的编址方案有字地址和字节地址两种：

字地址

0	12	34	56	78
4				

若字地址为 **高字节** 地址，则有：

字地址

0	12	34	56	78
4	4	5	6	7

IBM 370：32位字长

大端：优先存高字节

(2) 小端（优先存低字节）

设存储字长32位，存储器的编址方案有字地址和字节地址两种：

字地址

0	12	34	56	78
4				

若字地址为 **低字节** 地址，则有：

字地址

0	12	34	56	78
4	7	6	5	4

Intel 80x86：32位字长

小端：优先存低字节

三、寻址方式

确定本条指令的操作数地址；下一条欲执行指令的指令地址。

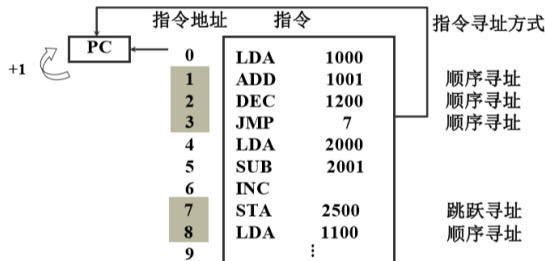
1、指令寻址

7.3.1 指令寻址

根据编址方式和指令长度不同，1是不确定的数。

顺序寻址 $(PC) + 1 \rightarrow PC$

跳跃寻址 由转移指令指出



2、数据寻址

操作码	寻址特征	形式地址 A
-----	------	--------

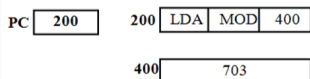
形式地址A 指令字中的地址

有效地址EA 操作数的真实地址

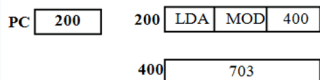
约定 指令字长=存储字长=机器字长

(1) 立即寻址 (获取操作数最快)

假设存储器按字编址，现有单字长指令存放在内存200号单元，该指令执行操作是将操作数调入累加器，若操作数的寻址方式为立即寻址，则累加器中的值为 400。指令执行完成后PC的值为 201。



假设存储器按字节编址，存储字长为32位。现有单字长指令存放在内存200号单元，该指令执行操作是将操作数调入累加器，若操作数的寻址方式为立即寻址，则累加器中的值为 400。指令执行完成后PC的值为 204。

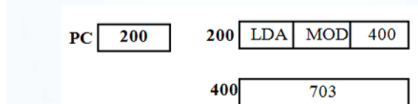


(2) 直接寻址

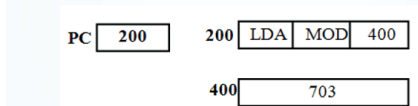
指令中地址码所给出的是操作数的有效地址，采

用直接寻址。

假设存储器按字编址，现有单字长指令存放在内存200号单元，该指令执行操作是将操作数调入累加器，若操作数的寻址方式为直接寻址，则累加器中的值为 703。指令执行完成后PC的值为 201。



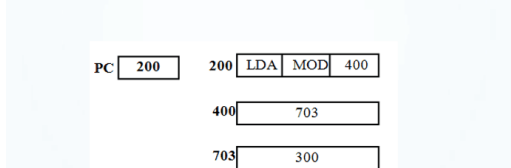
假设存储器按字节编址，存储字长为32位。现有单字长指令存放在内存200号单元，该指令执行操作是将操作数调入累加器，若操作数的寻址方式为直接寻址，则累加器中的值为 703。指令执行完成后PC的值为 204。



(3) 间接寻址

有效地址由形式地址间接提供。

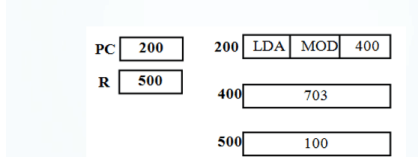
假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为间接寻址，则累加器中的值为 300。指令执行完成后PC的值为 201。



(4) 寄存器寻址

操作数在寄存器中的寻址 方式称为寄存器直接寻址。

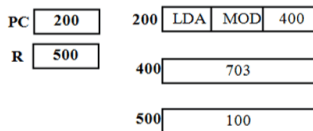
假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为寄存器寻址，则累加器中的值为 500。指令执行完成后PC的值为 201。



(5) 寄存器间接寻址

有效地址在寄存器中，操作数在存储器中，执行阶段访存；便于编制循环程序。

假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为寄存器间接寻址，则累加器中的值为 100。指令执行完成后PC的值为 201。



(6) 偏移寻址

基址寻址: $EA = (\text{基址}R) + A$ ，基址R不变，A可变

变址寻址: $EA = (\text{变址}R) + A$ ，A不变，变址R可变

相对寻址: $EA = (PC) + A$

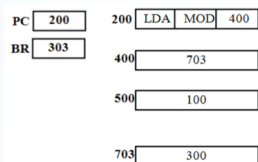
基址寻址的操作数有效地址EA是 $(BR) + A$ 。

变址寻址的操作数有效地址EA是 $(X) + A$ 。

相对寻址的操作数有效地址EA是 $(PC) + A$ 。

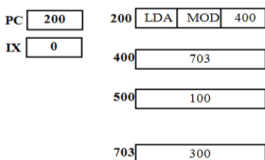
假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为基址寻址，则累加器中的值为 300。

指令执行完成后PC的值为 201。

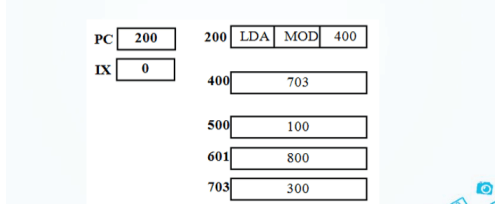


假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为变址寻址，则累加器中的值为 703。

指令执行完成后PC的值为 201。



假设单字长指令，执行操作是将操作数调入累加器，若操作数的寻址方式为相对寻址，则累加器中的值为 800。
指令执行完成后PC的值为 201。



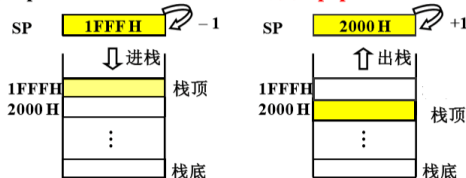
(7) 堆栈寻址

(1) 堆栈的特点

堆栈 { 硬堆栈 多个寄存器
 软堆栈 指定的存储空间

先进后出（一个入出口） 栈顶地址由 SP 指出

进栈 (push) (SP) - 1 → SP 出栈 (pop) (SP) + 1 → SP



(3) SP 的修改与主存编址方法有关

① 按字编址 { 进栈 (SP) - 1 → SP
 出栈 (SP) + 1 → SP

② 按字节编址

存储字长 16 位 { 进栈 (SP) - 2 → SP
 出栈 (SP) + 2 → SP

存储字长 32 位 { 进栈 (SP) - 4 → SP
 出栈 (SP) + 4 → SP

(1) CALL指令被读取前。

例7.3 一条**双字长直接寻址**的子程序调用指令，第一个字为操作码和寻址特征，第二个字为地址码**5000H**。假设PC当前的值为**2000H**，SP的内容为**0100H**，栈顶内容为**2746H**，存储器按**字节**编址，而且进栈操作是先执行 $(SP)-\Delta \rightarrow SP$ ，后存入数据。试回答下列几种情况下，PC、SP及栈顶的内容各是多少？

解：(1) CALL指令被读取前。

PC的值为2000H

SP的内容为0100H

栈顶内容为2746H



(2) CALL指令被执行之后

PC 5000H

SP 00FEH

解：CALL指令被取出之后，

PC=2004H。

调用子程序之前需要保护现场，即需要将程序断点地址（**2004H**）进栈保存。即栈顶内容为2004H。

此时 $SP=SP-2=0100H-2=00FEH$

PC的内容被更新为子程序的入口地址5000H



(3) 子程序返回后

PC 2004H

SP 0100H

解：栈顶内容2004H出栈送给PC。即恢复现场

此时 $SP=SP+2=00FEH+2=0100H$

栈顶内容为2746H。

字长?位=16位	
	字长
00FE H	2004H
0100 H	2746H
	⋮
2000 H	操作码 寻址特征
2002 H	5000H
2004 H	
	⋮
5000 H	子程序